

Act Des - Componentes en Mi CV profesional con Angular

Parte del proyecto “Mi CV profesional con Angular” que ya tienes terminado y funcionando. No crees un proyecto nuevo.

El objetivo de esta actividad es refactorizar la aplicación sustituyendo las diferentes zonas del layout por componentes Angular, sin modificar el resultado visual obtenido en la actividad anterior.

La aplicación original estaba construida aproximadamente con la siguiente estructura:

```
<header> ... </header>
<div class="main">
<aside> ... </aside>
<main> ... </main>
</div>
<footer> ... </footer>
```

En esta actividad cada una de estas responsabilidades deberá pasar a estar representada por un componente Angular.

No debes perder el layout realizado mediante CSS Flexbox ni cambiar el contenido del CV.

Crear rama git para este desarrollo

A partir del último commit de la actividad anterior de CV con angular crea una rama git nueva y sincronízala en github. Verifica que estás en la nueva rama. Y sobre esta rama realiza la refactorización del código que se solicita.

Comandos

```
git checkout -b refactor-componentes
git push -u origin refactor-componentes
git branch (la rama activa aparece marcada con *)
```

Componentes que debes crear

La aplicación debe quedar dividida, como mínimo, en los siguientes componentes:

- HeaderComponent → sustituirá a la cabecera <header>.
- MainLayoutComponent → sustituirá al contenedor <div class="main"> y será responsable del layout Flexbox central.
- AsideComponent → sustituirá a la sección lateral <aside>.
- MainComponent → sustituirá al contenido principal <main>.

- FooterComponent → sustituirá al pie <footer>.

Puedes organizar estos componentes dentro de una carpeta components del proyecto.

Recuerda la sintaxis general de Angular CLI para generar un componente:

ng generate component ruta/nombre-componente

También puedes utilizar su forma abreviada:

ng g c ruta/nombre-componente

Genera el componente encargado de la cabecera.

Comando utilizado:

```
ng g c components/header
```

Genera el componente encargado del contenedor central de la aplicación, que reemplazará a div.main.

Comando utilizado:

```
ng g c components/main-layout
```

Genera el componente encargado de la sección lateral izquierda.

Comando utilizado:

```
ng g c components/aside
```

Genera el componente encargado del contenido principal del CV.

Comando utilizado:

```
ng g c components/main
```

Genera el componente encargado del pie de página.

Comando utilizado:

```
ng g c components/footer
```

Comprueba la estructura de carpetas creada y anota los ficheros generados para uno de los componentes.

Ficheros generados:

```
En src/app/components/header/ se han generado:  
header.component.ts | header.component.html | header.component.css |  
header.component.spec.ts
```

Refactorización del componente raíz

El componente raíz ya no debe contener directamente toda la estructura HTML del CV. Su responsabilidad será componer las grandes zonas de la aplicación.

Al finalizar la refactorización, app.component.html debe contener únicamente los componentes de primer nivel de la aplicación: cabecera, zona central y pie.

Escribe cómo ha quedado tu app.component.html:

```
<app-header></app-header>  
<app-main-layout></app-main-layout>  
<app-footer></app-footer>
```

TIP:

en una aplicación basada en componentes, el selector de cada componente es la etiqueta que utilizas para incluirlo dentro de otro componente.

Por ejemplo, si un componente tuviera el selector app-ejemplo, se utilizaría así:

```
<app-ejemplo></app-ejemplo>
```

Componente del layout central

MainLayoutComponent sustituye al antiguo div.main. Dentro de él debes colocar los componentes que representan la sección lateral y el contenido principal.

Escribe la plantilla HTML que has utilizado en MainLayoutComponent:

```
<app-aside></app-aside>  
<app-main></app-main>
```

El punto importante de esta parte es mantener el comportamiento Flexbox que tenía el contenedor .main.

Antes de componentizar:

```
.main {  
  display: flex;  
}
```

El contenedor `.main` era el flex container y los elementos `aside` y `main` eran sus flex items.

Después de componentizar, puedes hacer que el propio componente encargado del layout se comporte como flex container utilizando `:host` en su hoja de estilos.

TIP:

`:host` permite aplicar estilos al elemento HTML que representa al propio componente Angular.

```
:host {  
  display: flex;  
}
```

Explica por qué es necesario mantener `display: flex` en el componente que sustituye al antiguo `div.main`:

El flex container es quien reparte el espacio entre sus hijos directos. `<app-main-layout>` es un elemento propio del componente y por defecto es inline, sin `display: flex` en su `:host`, `app-aside` y `app-main` se apilarían uno encima de otro y no se aplicaría el reparto lateral/principal. Con `:host { display: flex }` el componente sustituye al antiguo `div.main`.

Los componentes también son elementos del layout

Después de la refactorización, los hijos directos del contenedor Flexbox ya no son los elementos HTML `aside` y `main`. Ahora los flex items son los elementos de los componentes Angular que los contienen.

De forma conceptual, el navegador verá una estructura similar a:

```
<app-main-layout>  
<app-aside></app-aside>  
<app-main></app-main>  
</app-main-layout>
```

Por este motivo, si antes aplicabas el ancho directamente a `aside` y `main`, puede que el layout deje de respetar el reparto 30 % / 70 %.

TIP:

puedes aplicar las propiedades del elemento Flexbox al `:host` de cada componente.

Ejemplo orientativo para el componente lateral:

```
:host {  
  display: block;  
  flex: 0 0 30%;  
}
```

Ejemplo orientativo para el componente principal:

```
:host {  
  display: block;  
  flex: 1;  
  min-width: 0;  
}
```

No es obligatorio utilizar exactamente estas reglas. Puedes mantener tu propia solución CSS siempre que el resultado conserve el layout de la actividad anterior.

Escribe las reglas principales que has utilizado para conseguir el reparto entre AsideComponent y MainComponent:

```
MainLayout :host { display: flex; align-items: stretch; }  
Aside :host { display: block; flex: 0 0 28%; }  
Main :host { display: block; flex: 1; min-width: 0; }  
Asi app-aside ocupa ~28% y app-main el resto, como antes.
```

Cabecera y pie

Mueve el HTML de la cabecera al template de HeaderComponent y el HTML del pie al template de FooterComponent.

Al convertir estas zonas en componentes, recuerda que `<app-header>` y `<app-footer>` son ahora elementos HTML propios.

TIP:

si necesitas que ocupen todo el ancho de su contenedor, puedes hacer que el host del componente se comporte como un elemento de bloque:

```
:host {  
  display: block;  
}
```

No copies todos los estilos en app.component.css. Cada componente debe conservar en su propia hoja CSS los estilos relacionados con su responsabilidad visual.

Distribución de los datos del CV

En la actividad anterior los datos del CV estaban definidos en `app.component.ts` y se mostraban mediante interpolación en `app.component.html`.

Ahora las plantillas han cambiado de componente. Una propiedad definida en AppComponent no está disponible automáticamente dentro del template de HeaderComponent, AsideComponent, MainComponent o FooterComponent.

Para esta actividad, mueve los datos al componente que los muestra. Por ejemplo:

- HeaderComponent → nombre, profesión y descripción principal.
- AsideComponent → ciudad, teléfono, correo, GitHub e idiomas.
- MainComponent → descripción, experiencia, formación y tecnologías.

- FooterComponent → texto del pie y fecha.

Anota qué propiedades has trasladado a cada componente:

HeaderComponent: nombre, profesion, fotoPerfil.
 AsideComponent: ciudad, numero, email, github, idiomas y habilidades (lista de lenguajes).
 MainComponent: sobreMi, experiencias y formacion.
 FooterComponent: textoPie y fechaActual.
 AppComponent: ya no guarda datos (solo compone).

TIP: la interpolación {{ propiedad }} busca la propiedad en la clase TypeScript del mismo componente al que pertenece el template.

Si posteriormente estudias comunicación entre componentes, estos datos también podrían recibirse desde un componente padre mediante inputs. En esta actividad no es necesario hacerlo.

Imports de los componentes standalone

Si estás trabajando con componentes standalone, cada componente debe importar aquello que utiliza directamente en su template.

Revisa especialmente los elementos utilizados en la actividad anterior:

- El componente que utilice NgOptimizedImage debe realizar el import correspondiente.
- El componente que formatee la fecha mediante DatePipe debe disponer de DatePipe en sus imports.
- El componente padre debe importar los componentes cuyos selectores utiliza en su template.
- MainLayoutComponent debe conocer los componentes que utiliza para la zona lateral y el contenido principal.

Indica qué imports has tenido que modificar o trasladar tras realizar la componentización:

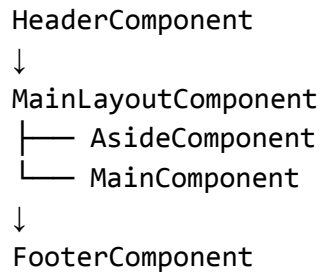
HeaderComponent: NgOptimizedImage (foto de perfil).
 FooterComponent: DatePipe (se traslada desde AppComponent).
 MainLayoutComponent: AsideComponent y MainComponent.
 AppComponent: HeaderComponent, MainLayoutComponent y FooterComponent (se quitan DatePipe y NgOptimizedImage).
 Aside y Main no necesitan imports (@for es sintaxis propia).

Estructura final esperada

La organización de responsabilidades debe quedar aproximadamente de la siguiente forma:

AppComponent

↓



Y el reparto de responsabilidades debe ser:

- AppComponent → composición general de la aplicación.
- HeaderComponent → presentación del candidato.
- MainLayoutComponent → distribución Flexbox de la zona central.
- AsideComponent → información personal y de contacto.
- MainComponent → información profesional del CV.
- FooterComponent → texto final y fecha.

Comprobación del resultado

Al finalizar, la aplicación debe conservar el mismo aspecto visual que tenía antes de la refactorización.

Comprueba especialmente que:

- la cabecera ocupa el ancho previsto;
- la zona central continúa utilizando Flexbox;
- la sección lateral mantiene aproximadamente un 25 % o 30 % del ancho;
- el contenido principal utiliza el espacio restante;
- el pie aparece en la parte inferior y mantiene su alineación;
- los datos continúan mostrándose mediante interpolación;
- el array de tecnologías continúa representándose correctamente;
- la imagen continúa funcionando con NgOptimizedImage;
- la fecha continúa formateándose mediante DatePipe;
- no aparecen errores en la consola del navegador.

TIP de depuración del layout

Si después de crear los componentes el lateral aparece encima del contenido principal, utiliza las herramientas de desarrollo del navegador e inspecciona qué elementos son hijos directos del contenedor que tiene `display: flex`.

Recuerda: Flexbox distribuye a los hijos directos. Después de componentizar, esos hijos pueden ser `app-aside` y `app-main` en lugar de `aside` y `main`.

Describe un problema de layout que hayas encontrado durante la actividad y cómo lo has solucionado:

Tras crear los componentes, el lateral aparecía encima del contenido principal. Causa: los hijos directos del contenedor eran `app-aside` y `app-main`, y `app-main-layout` (inline por defecto) no era flex container, además los anchos estaban en las clases antiguas.
Solución: `:host { display: flex }` en `MainLayout`, `:host { display: block; flex: 0 0 28% }` en `Aside` y `:host { display: block; flex: 1; min-width: 0 }` en `Main`.

Entrega

Entrega el proyecto Angular completo mediante enlace al repositorio remoto de GitHub, igual que en la actividad anterior junto con este documento con los cuadros de respuesta rellenados.

Antes de realizar la entrega comprueba que la aplicación puede instalarse y ejecutarse correctamente.

Comandos de comprobación utilizados:

```
git clone <url-repo> && cd mi-cv  
npm install | ng serve -o | ng build
```

No se valorará únicamente que el CV conserve su aspecto visual.

El objetivo principal de esta actividad es comprobar que sabes transformar un layout monolítico en una estructura de componentes Angular manteniendo correctamente las responsabilidades de HTML, TypeScript y CSS.

Debes poder explicar la siguiente relación:

```
AppComponent  
↓ composición  
Componentes  
↓ templates independientes  
Datos en el componente correspondiente  
↓  
Estilos CSS encapsulados  
↓  
:host y Flexbox  
↓  
mismo renderizado final
```